



PDF Download
3772288.pdf
18 December 2025
Total Citations: 0
Total Downloads: 56

 Latest updates: <https://dl.acm.org/doi/10.1145/3772288>

RESEARCH-ARTICLE

The ArborX Library: Version 2.0

ANDREY PROKOPENKO, Oak Ridge National Laboratory, Oak Ridge, TN, United States

DANIEL ARNDT, Oak Ridge National Laboratory, Oak Ridge, TN, United States

DAMIEN LEBRUN-GRANDIE, Oak Ridge National Laboratory, Oak Ridge, TN, United States

BRUNO TURCK SIN, Oak Ridge National Laboratory, Oak Ridge, TN, United States

Open Access Support provided by:

Oak Ridge National Laboratory

Published: 12 December 2025

Online AM: 21 October 2025

Accepted: 15 October 2025

Revised: 03 October 2025

Received: 28 July 2025

[Citation in BibTeX format](#)

The ArborX Library: Version 2.0

ANDREY PROKOPENKO, DANIEL ARNDT, DAMIEN LEBRUN-GRANDIÉ, and BRUNO TURCK SIN, Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA

This article provides an overview of the 2.0 release of the ArborX library, a performance portable geometric search library based on Kokkos. We describe the major changes in ArborX 2.0 including a new interface for the library to support a wider range of user problems, new search data structures (brute force and distributed), support for user functions to be executed on the results (callbacks), and an expanded set of the supported algorithms (ray tracing and clustering).

CCS Concepts: • **Mathematics of computing** → **Mathematical software**;

Additional Key Words and Phrases: geometric search, Kokkos, GPU, clustering

ACM Reference format:

Andrey Prokopenko, Daniel Arndt, Damien Lebrun-Grandié, and Bruno Turcksin. 2025. The ArborX Library: Version 2.0. *ACM Trans. Math. Softw.* 51, 4, Article 30 (December 2025), 10 pages.
<https://doi.org/10.1145/3772288>

1 Overview

ARBORX is a performance portable geometric search library. ARBORX version 2.0 was released 16 April 2025. This article provides an overview of the new features developed in ARBORX since the original paper [Lebrun-Grandié et al., 2020] published in 2020, and serves as a citable reference for the ARBORX software library version 2.0. It is available for free under the terms of *BSD 3-Clause* license. It is available on GitHub at <https://github.com/arborex/ArborX>.

The major changes in ARBORX since Lebrun-Grandié et al. [2020] are:

- New interface for search indexes (Section 2.1).
- Support for user functions called on a match (callbacks) (Section 2.2).
- Support for distributed search using MPI (Section 2.3).
- New clustering algorithms (Section 2.4).
- Support for ray-tracing (Section 2.5).
- Multiple performance improvements (Section 2.6).

This research was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration.

Authors' Contact Information: Andrey Prokopenko (corresponding author), Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA; e-mail: prokopenkoav@ornl.gov; Daniel Arndt, Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA; e-mail: arndtd@ornl.gov; Damien Lebrun-Grandié, Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA; e-mail: lebrungrandt@ornl.gov; Bruno Turcksin, Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA; e-mail: turcksinbr@ornl.gov.



This work is licensed under Creative Commons Attribution International 4.0.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7295/2025/12-ART30

<https://doi.org/10.1145/3772288>

While the major changes are discussed in detail in Section 2, there are several other noteworthy changes included in the 2.0 release, which we briefly outline here:

- Modernization of the code and build system. The minimum requirements for ARBORX are now C++20, CMake 3.22, and Kokkos 4.5.
- New brute-force search structure.
- Wider dimensionality and precision support for geometries. Geometries now support different data dimensions (1–10) and floating point precisions.
- Increased support for AMD and Intel GPUs (through the developments in Kokkos HIP and SYCL backends).
- Expanded list of supported geometries. In addition to previously supported points, **axis-aligned bounding boxes (AABBs)** and spheres, ARBORX now supports ***k*-discrete oriented polytopes (*k*-DOPs)** [Klosowski et al., 1998], triangles, rays, tetrahedrons, and segments.
- New implementation of the moving least squares algorithm [Quaranta et al., 2005] as part of the interpolation subpackage.

2 Major Changes to the Library

This release contains a number of significant changes, which we will discuss in this section. A number of these changes required breaking backwards compatibility.

2.1 Updates to Interface

2.1.1 Original Interface (API v1). The main search data structure (index) in ARBORX is **bounding volume hierarchy (BVH)**. Its original interface looked like the following:

```
template <typename MemorySpace>
class BVH {
public:
    using memory_space = MemorySpace;
    using size_type = typename MemorySpace::size_type;
    using bounding_volume_type = ArborX::Box;

    template <typename Primitives>
    BVH(Primitives const &primitives);

    template <typename Predicates>
    void query(Predicates const &predicates, Kokkos::View<int *, MemorySpace> &indices,
              Kokkos::View<int *, MemorySpace> &offset) const;

    KOKKOS_FUNCTION size_type size() const;
    KOKKOS_FUNCTION bool empty() const;
    KOKKOS_FUNCTION bounding_volume_type bounds() const;
};
```

The following example shows how the interface was used in practice:

```
// Create the View for the bounding boxes
Kokkos::View<ArborX::Box*, MemorySpace> boxes("boxes", num_boxes);
// Fill in the bounding boxes
...
// Create the bounding volume hierarchy
ArborX::BVH<MemorySpace> bvh(boxes);
// Create the View for the spatial queries
Kokkos::View<ArborX::Within *, MemorySpace> queries("queries", num_queries);
```

```
// Fill in the queries
...
// Perform the search
Kokkos::View<int*, MemorySpace> offsets("offsets", 0);
Kokkos::View<int*, MemorySpace> indices("indices", 0);
bvh.query(queries, indices, offsets);
```

First, a user constructs bounding boxes for the data, and calls the constructor. Next, the queries are built. Each query corresponds to a pair of a query point and a number of neighbors to be found (nearest query), or a pair of a query point and a radius (spatial query). Then, two views are allocated to store the results, indices (the indices of the bounding boxes that satisfy the queries) and offsets (the offsets in indices for each query).¹ Finally, the search is done by invoking BVH's query function.

2.1.2 Limitations of the Original Interface. The original interface was designed to be easy to use while providing some limited flexibility. With a growth in the number of users, and proliferation of desired search tasks for different applications, it became important to reduce the existing limitations, which we briefly discuss below.

Hardcoded Dimensionality. ARBORX only supported 3D objects. It also supported 2D objects by treating them as 3D through setting $z = 0$, at the cost of extra memory storage and slower traversal.

Lack of Execution Spaces. The interface did not allow users to pass execution spaces as an argument. Instead, it used the default Kokkos execution space. This made it impossible to run multiple searches simultaneously, or running the search in parallel with other user kernels. This also made any fencing global for a device, not a single stream.

Lack of Fine Nearest Neighbor Search. Users were able to only expose the axis-aligned boxes to ARBORX through the interface. When searching for the nearest objects, ArborX only considered distances to those boxes, and not to the user data. This meant that for user geometries other than points and boxes, the results produced by ARBORX corresponded to a coarse search.

Hardcoded Bounding Volume Type. ARBORX only supported AABBs as bounding volume. There are many other bounding volumes used in practice and more efficient for certain problems, such as oriented bounding boxes and k -DOPs [Klosowski et al., 1998], and others. The choice of a bounding volume is particularly important for distributed search queries, when this directly impacts the amount of network communication.

No Support for User Functions (Callbacks). Users are often interested in performing some operation on the results of each query. For example, they may be only interested in the number of neighbors, average distance, or updating some quantity. The results produced as a pair of (offsets, indices) serve only as an intermediate step for that, at the cost of extra memory reads and writes. Moreover, the number of the results may exceed the memory capacity for particularly dense problems. In the distributed search using MPI, it additionally cuts down on the number of communication rounds and the amount of transferred data between different ranks. The original interface did not allow users to specify an operation to execute on the results without storing them.

2.1.3 New Interface (API v2). To address the mentioned limitations, the interface of ARBORX was reworked in version 2.0. The new interface of the search indexes was strongly inspired by Boost.Geometry.Index [Gehrels et al., 2025]. An index may now store user-provided objects of any

¹This format is similar to that of compressed sparse row format that is commonly used to store sparse matrices.

type `Value`, as long as ARBORX knows how to extract a geometric object of type *Indexable*. This extraction is done with a helper functor `IndexableGetter` that can be specified by a user. This allows ARBORX to behave like a container while also providing a search index over the provided data.

The current interface looks as follows:

```
template <typename MemorySpace, typename Value, typename IndexableGetter, typename BoundingBox>
class BVH {
public:
    using memory_space = MemorySpace;
    using size_type = typename MemorySpace::size_type;
    using bounding_volume_type = BoundingBox;
    using value_type = Value;

    template <typename ExecutionSpace, typename Values>
    BVH(ExecutionSpace const &space, Values const &values,
        IndexableGetter const &indexable_getter = IndexableGetter());

    template <typename ExecutionSpace, typename Predicates, typename Callback>
    void query(ExecutionSpace const &space, Predicates const &predicates,
        Callback const &callback); // (1)
    template <typename ExecutionSpace, typename Predicates, typename Callback>
    void query(ExecutionSpace const &space, Predicates const &predicates,
        Callback const &callback,
        Kokkos::View<ValueLike *, MemorySpace> &values,
        Kokkos::View<int *, MemorySpace> &offset) const; // (2)
    template <typename ExecutionSpace, typename Predicates>
    void query(ExecutionSpace const &space, Predicates const &predicates,
        Kokkos::View<Value *, MemorySpace> &values,
        Kokkos::View<int *, MemorySpace> &offset) const; // (3)

    KOKKOS_FUNCTION size_type size() const noexcept;
    KOKKOS_FUNCTION bool empty() const noexcept;
    bounding_volume_type bounds() const noexcept;
};
```

The major differences with the original interface are:

- Updated template arguments. A search index is now templated on a Kokkos memory space, stored value type, indexable getter, and bounding volume.
- Taking in a Kokkos execution space for both constructor and queries, allowing multiple searches to be performed in parallel.
- Introduction of an indexable getter argument in the constructor.
- Three different query calls:
 - (1) pure callback query that executes an operation on every match but does not store the results;
 - (2) a callback query that does an operation on a match and stores the result of that operation. As this operation may change the type, the type of the results stored in the values array may be different from `Value` type; and
 - (3) a simple storage query that is similar to APIv1.

The `Values` in the constructor can be provided using the `ArborX::AccessTraits` mechanism as before. Together with callbacks (described in Section 2.2), this provides a flexible interface, supporting many more of the use cases.

For example, the following code shows how to use the new interface:

```
// Create the View for the bounding boxes
Kokkos::View<ArborX::Box<3> *, MemorySpace>
    boxes("boxes", num_boxes);
// Fill in the bounding boxes
...
// Create an execution space
Kokkos::DefaultExecutionSpace space;
// Create the bounding volume hierarchy
ArborX::BVH const bvh(space, boxes);
// Create the View for the spatial queries
Kokkos::View<decltype(ArborX::intersects(ArborX::Point<3>{})) *, MemorySpace>
    queries("queries", num_queries);
// Fill in the queries
...
// Perform the search
Kokkos::View<int*, MemorySpace> offsets("offsets", 0);
Kokkos::View<ArborX::Box<3> *, MemorySpace> values("values", 0);
bvh.query(space, queries, values, offsets);
```

As before, a user starts by constructing bounding boxes for the data. Note that the bounding boxes are now templated on the spatial dimension. Next, BVH's constructor is called. Compared to the old API, the user needs to provide a Kokkos execution space that encapsulates the execution resources to use to perform the work. Next, the queries are built. Then, two views are allocated to store the results, values (the bounding boxes that satisfy the queries) and offsets (the offsets in values for each query). Finally, the search is performed by invoking the BVH's query function. The main difference with the old API is that query returns the objects used to build the BVH object instead of their indices.

2.2 Callbacks

Callbacks are functors that are called on values that satisfy a predicate. Two different callbacks are supported. The first one does not produce any output (*pure* callback):

```
struct Callback {
    template<typename Predicate, typename Value>
    KOKKOS_FUNCTION
    RT operator()(Predicate const &predicate, Value const &value) const;
};
```

Here, RT is the return type, which can either be void, or a special type indicating early traversal termination.

The second callback type allows to produce output:

```
struct CallbackWithOutput {
    template<typename Predicate, typename Value, typename OutputFunction>
    KOKKOS_FUNCTION
    void operator()(Predicate const &predicate, Value const &value,
        OutputFunction const &output) const;
};
```

Users may use this callback combined with a general query (2) to finely control the stored output of each query. A user may compute and store any number of values for each positive match. These values may differ from the data stored in the index. For example, a user may desire to find all mesh

elements that contain a given degree of freedom, interpolate a field from the found element, and store the interpolated value as an output.

The following example counts the number of results for each query up to a specified limit N . Once the number of results for a given query reaches the limit, the traversal for that query is terminated.

```
template<typename Counts>
struct CountUpToNCallback {
    Counts _counts;
    int _n;
    template<typename Predicate, typename Value>
    void operator()(Predicate const &predicate, Value const &) const {
        int const i = ArborX::getData(query);
        if (_counts(i)++ >= _n)
            return ArborX::CallbackTreeTraversalControl::early_exit;
        return ArborX::CallbackTreeTraversalControl::normal_continuation;
    }
};
// Create an execution space
Kokkos::DefaultExecutionSpace space;
// Construct a hierarchy and queries
ArborX::BVH const bvh(space, ...);
Kokkos::View<ArborX::Within<...> *, MemorySpace> queries(...);
// Perform the search
constexpr int N = 10;
Kokkos::View<int*, MemorySpace> counts("counts", queries.size());
bvh.query(space, ArborX::attach_indices(queries), CountUpToNCallback{counts, N});
```

Here, we used the ARBORX helper function `attach_indices` to attach indices to a query. In general, ARBORX allows attaching arbitrary data to each query. The attached data is then obtained inside the callback through `getData()` call. We store the counts in a temporary view. As queries are not shared among threads, we do not need atomic operations when increasing the count number for each query. The callback returns the `early_termination` value once the threshold is reached. This value terminates the traversal for that query.

2.3 Distributed Search

ARBORX now provides a distributed search index `ArborX::DistributedTree`, with an interface closely following `ArborX::BVH` only adding an `MPI_Comm` constructor argument.

The distributed search supports both spatial and nearest queries. Each processor constructs a local search index based on the provided data, and then a top level index providing an overview of the local bounding boxes. During the search, each processor provides a set of local queries that are then communicated to the processors that have the necessary data. The result is returned to the processor originating a query.

Distributed search fully supports callbacks. Callbacks are executed on the processor owning the data that satisfies a query. This allows to reduce communication required to execute an operation by avoiding crossing the network multiple times to execute operations. One could consider, for example, a distributed finite element mesh and asking to interpolate a solution on that mesh into a set of query points. Using callbacks achieves this without a need to communicate mesh elements among MPI ranks.

Distributed search can also take advantage of the GPU-aware MPI, if present, through the use of `ARBORX_ENABLE_GPU_AWARE_MPI` configuration option. When enabled, the data stays resident to accelerator memory instead of being transferred to host for communications.

The following example show a procedure to find N closest points in a distributed setting:

```

struct PairIndexRank {
    unsigned index;
    int rank;
};

struct CallbackWithRank
{
    int _rank;
    template <typename Predicate, typename Value, typename Output>
    KOKKOS_FUNCTION
    void operator()(Predicate const &, ArborX::PairValueIndex<Value> const &value,
                    Output const &out) const {
        out({.index = value.index, .rank = _rank});
    }
};

// Create an execution space
ExecutionSpace exec;
// Set up an MPI communicator
MPI_Comm comm = MPI_COMM_WORLD;
// Find local rank
int comm_rank;
MPI_Comm_rank(comm, &comm_rank);
// Construct local data on each MPI rank
Kokkos::View<ArborX::Point<3> *, MemorySpace> points("points", n);
...
// Build a distributed tree
ArborX::DistributedTree tree(comm, exec, ArborX::attach_indices(points));

// Find N closest points for each local point and store them together with
// the rank they belong to
constexpr int N = 3;
Kokkos::View<PairIndexRank *, MemorySpace> values("indices_with_ranks", 0);
Kokkos::View<int *, MemorySpace> offsets("offsets", 0);
tree.query(exec, ArborX::make_nearest(points_device, N),
           CallbackWithRank{comm_rank}, values, offsets);

```

We start with constructing a set of local points and proceed with building a distributed index. As we mentioned before, the only difference in the distributed constructor's interface is the presence of `MPI_Comm` as the first argument. Internally, the provided communicator is duplicated so that ARBORX's internal communication is isolated from the user application. We attach the indices to the data, and then perform the search. We use another helper function here, `make_nearest`, to easily construct a set of nearest queries for the data. We also use a `CallbackWithRank` callback, as by default ARBORX only returns the data stored in the tree. The result is then stored as pairs of an index with the rank it was found on.

2.4 Clustering Algorithms

ARBORX now provides access to several clustering algorithms:

- ***Density-based spatial clustering of applications with noise (DBSCAN)***. DBSCAN [Ester et al., 1996] is a density-based clustering algorithm. Compared to other techniques like k -means, it can deal with clusters of arbitrary shapes and does not require knowing the number of clusters *a priori*.

ARBORX provides two implementations of the algorithm depending on the sparsity of the data (with respect to the DBSCAN's ϵ parameter). A user may choose to use either FDBSCAN (for very sparse data) or FDBSCAN-DenseBox (for data with dense regions) algorithms. Both algorithms are implemented to run efficiently on both CPUs and GPUs, and have been used in production for large scale cosmology problems [Prokopenko et al., 2025]. For more information, see Prokopenko et al., [2023a].

The work to implement DBSCAN algorithm in the distributed setting is ongoing.

- **Euclidean minimum spanning tree (EMST)**. EMST computes the minimum spanning tree of the distance graph of a set of points, i.e., a graph where each pair of vertices are connected by an edge of weight equal to the distance between them. EMST also serves as a basis for the HDBSCAN* algorithm [Campello et al., 2015]. ARBORX provides a new implementation of the algorithm suitable for GPUs. For more information, see Prokopenko et al. [2023b].

As a simple example, consider running DBSCAN algorithm for a given set of 2D points:

```
// Create an execution space
ExecutionSpace exec;
// Construct points
Kokkos::View<ArborX::Point<2> *, MemorySpace> points("points", n);
...
// Run DBSCAN
constexpr float eps = 1.5;
constexpr int minPts = 5;
auto labels = ArborX::dbscan(exec, points, eps, minPts);
```

The result is stored in a 1D integer array `labels`. If a given label is equal to `-1`, the corresponding point is a noise point. Otherwise, the label corresponds to a cluster index.

2.5 Ray Tracing Support

ARBORX now supports ray tracing. While there are multiple ways to define a ray, an `ArborX::Ray` is defined by its origin and its direction. Ray tracing is currently supported by a limited set of geometries: a box, a triangle and a sphere. ARBORX provides three kinds of predicates for ray tracing:

- `nearest`: This predicate finds the first k geometric objects that the rays intersect. One could consider the rays to be absorbed after k collisions. The case $k = 1$ corresponds to finding the closest geometric object that the ray intersects.
- `intersect`: This predicate finds all geometric objects that the rays intersect. It is equivalent to the nearest predicate where k is infinity. This can be likened to the rays intersecting perfectly transparent objects.
- `ordered_intersect`: This predicate is similar to `intersect` but the intersected objects are ordered as if a physical ray would encounter them. This can be useful when a ray is depositing energy through a medium and it is necessary to keep track of the history of the intersections.

The nearest and intersect predicates are also supported in the case of distributed ray tracing where the geometric objects and the rays exist on different MPI ranks.

2.6 Performance Improvements

ARBORX's internal implementation of the BVH was modified significantly in this release, dramatically boosting its performance. We highlight the following changes among many others:

- Morton codes used during the construction changed from 32-bit to 64-bit by default.

Table 1. Performance Comparison of Old and New ARBORX Versions on a $\approx 37M$ Cosmology Dataset on Nvidia A100 GPU

	Old (no Thrust)	Old (with Thrust)	New
Construction	17.006	0.021	0.018
Query	18.619	1.272	0.257
Query (count)	-	-	0.124

- The hierarchy construction algorithm switched from Karras [2012] to Apetrei [2014] algorithm.
- Spatial search is now performed using stackless algorithm [Prokopenko and Lebrun-Grandie, 2024].
- Search for pairs of objects now uses a special traversal algorithm (see Prokopenko et al., [2025]).
- Search can now be terminated early through the use of callbacks (see Section 2.2).
- Sorting in both construction and search now takes advantage of the vendor-specific libraries (Thrust for CUDA, rocThrust for HIP, and oneDPL for SYCL).

In Table 1, we compare the performance between two versions of ARBORX: the “old” version (hash 02f9a81, March 2020) and the “new” version (version 2.0, hash 041bf18e, April 2025), compiled against Kokkos 4.7,² using CUDA 12.4. For our experiments, we used a cosmology dataset from Section 4 in Prokopenko et al., [2025] containing $\approx 37M$ points. We ran the construction and search procedure (with radius $r = 0.042$) on a Nvidia A100 GPU. As we can see, using Thrust instead of the native Kokkos::BinSort algorithm is critical. Further $5\times$ improvements are coming from the stackless traversal algorithm as well as from storing less data (leaf nodes containing points now take 16 bytes instead of 32). When comparing old and new versions, all found results were stored. We also display the performance of the query when the results are simply counted using a pure callback. It shows an additional improvement of over $2\times$.

Acknowledgements

Other than the authors of this article, the following people contributed code to this release: Yohann Bosqued, Jared Crean, Ana Gainaru, Wenjun Ge, Christoph Junghans, Shihab Shahriar Khan, Phil Miller, and Tuan Pham. Their contributions are much appreciated!

References

- C. Apetrei. 2014. Fast and simple agglomerative LBVH construction. In *Computer Graphics and Visual Computing (CGVC)*. R. Borgo and W. Tang (Eds.), The Eurographics Association. DOI: <https://doi.org/10.2312/cgvc.20141206>
- R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander. 2015. Hierarchical density estimates for data clustering, visualization, and outlier detection. *ACM Transactions on Knowledge Discovery from Data* 10, 1, Article 5 (July 2015), 1–51. DOI: <https://doi.org/10.1145/2733381>
- M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96)*. AAAI Press, Portland, OR, 226–231.
- B. Gehrels, B. Lalande, M. Loskot, A. Wulkiewicz, M. Karavelas, and M. Fisikopoulos. 2025. Boost Geometry. Retrieved from https://www.boost.org/doc/libs/1_88_0/libs/geometry/doc/html/geometry/spatial_indexes/introduction.html
- T. Karras. 2012. Maximizing parallelism in the construction of BVHs, octrees, and k -d trees. In *Proceedings of the 4th ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics (EGGH-HPG '12)*. Eurographics Association, Goslar, 33–37. DOI: <https://doi.org/10.2312/EGGH/HPG12/033-037>

²The old version required minor patching to work with a new version of Kokkos.

- J. T. Klosowski, M. Held, J. S. B. Mitchell, H. Sowizral, and K. Zikan. 1998. Efficient collision detection using bounding volume hierarchies of k-DOPs. *IEEE Transactions on Visualization and Computer Graphics* 4, 1 (Jan. 1998), 21–36. DOI : <https://doi.org/10.1109/2945.675649>
- D. Lebrun-Grandié, A. Prokopenko, B. Turcksin, S. and R. Slattery. 2020. ArborX: A performance portable geometric search library. *ACM Transactions on Mathematical Software* 47, 1, Article 2 (Dec. 2020), 1–15. DOI : <https://doi.org/10.1145/3412558>
- A. Prokopenko, D. Arndt, D. Lebrun-Grandié, B. Turcksin, N. Frontiere, J. D. Emberson, and M. Buehlmann. 2025. Advances in ArborX to support exascale applications. *The International Journal of High Performance Computing Applications* 39, 1 (2025), 167–176. DOI : <https://doi.org/10.1177/10943420241298296>
- A. Prokopenko and D. Lebrun-Grandie. 2024. *Revising Apetrei's Bounding Volume Hierarchy Construction Algorithm to Allow Stackless Traversal*. Technical report ORNL/TM-2024/3259. Oak Ridge National Laboratory (ORNL), Oak Ridge, TN. DOI : <https://doi.org/10.2172/2301619>
- A. Prokopenko, D. Lebrun-Grandie, and D. Arndt. 2023. Fast tree-based algorithms for DBSCAN for low-dimensional data on GPUs. In *Proceedings of the 52nd International Conference on Parallel Processing (ICPP '23)*. ACM, New York, NY, 503–512. DOI : <https://doi.org/10.1145/3605573.3605594>
- A. Prokopenko, P. Sao, and D. Lebrun-Grandie. 2023. A single-tree algorithm to compute the Euclidean minimum spanning tree on GPUs. In *Proceedings of the 51st International Conference on Parallel Processing (ICPP '22)*. ACM, New York, NY, 1–10. DOI : <https://doi.org/10.1145/3545008.3546185>
- G. Quaranta, P. Masarati, and P. Mantegazza. 2005. A conservative mesh-free approach for fluid-structure interface problems. In *Proceedings of the International Conference for Coupled Problems in Science and Engineering*.

Received 28 July 2025; revised 3 October 2025; accepted 15 October 2025